The release candidate is passed to subsequent build jobs, each of which acts as a quality gateway. In the pipeline illustrated here, the quality gateways are the automated acceptance criteria and the performance tests. If the release candidate passes both of these gateways successfully, it can be deployed to the User Acceptance Testing (UAT) environment. Deployment to UAT and production is triggered manually—the process is still automated, but the team decides when the latest release candidate should be deployed.

In a more traditional release process, when the code is deemed ready, a special "release build" will produce a release candidate version of the application. This release candidate version will need to pass through a similar set of quality gateways before it can be released into production, but it will generally be rebuilt from source code at each stage.

In both cases, the build pipeline is typically made up of a series of quality gateways, which run different sorts of tests and checks. Common examples of quality gateways include

- A simple, fast build that compiles and runs the unit and integration tests (in that order). This is designed to provide feedback to developers. Using BDD naming styles makes it easier to troubleshoot problems when they do occur.
- A longer-running build job that runs the automated acceptance criteria. This build job also produces and publishes the living documentation.
- A build job that verifies code quality metrics such as code coverage or coding standards.
- Build jobs that verify performance or other nonfunctional requirements (discussed in section 9.4).

Many modern CI servers now support build pipelines to various degrees. Figure 12.6 illustrates the build pipeline illustrated in figure 12.5 running on a Jenkins server using the Build Pipeline plugin (https://wiki.jenkins-ci.org/display/JENKINS/Build+Pipeline+Plugin).

Teams practicing BDD are in a good position to implement continuous delivery strategies. Continuous delivery relies on an extremely high level of confidence in the automated test suite: if you can't trust the automated tests to verify your application, you need to introduce a manual testing step, or possibly several manual testing steps, into the pipeline. Many organizations do indeed have manual testing steps, but the number and length of these steps has a significant impact on the time it takes a given release candidate to get into production. The more confidence the team has in the automated tests, the less time needs to be spent on these manual testing stages.

This is one of the reasons that BDD practitioners place such a strong emphasis on communication and readability when writing automated acceptance scenarios. Testers need to be able to understand what the automated acceptance tests are verifying, and to trust that they're testing these aspects of the application effectively. Otherwise, they'll effectively duplicate the verification work done by the automated tests, which